

# Pseudo Random Number Generators

Giulio Beltramin

2026-10-09

## Sampling from the Uniform distribution

### Linear Congruential Generator (1958, Thomson & Rotenberg)

The Linear Congruential Generator is of the first Pseudo-Random Number Generators (PRNGs). It involves only very simple instructions.

Let  $m$  (modulus),  $\mu$  (multiplier),  $c$  (increment) and  $X_0$  (seed) integers.

The LCG iterates

$$X_{i+1} = (\mu X_i + c) \% m$$

Then  $X_j$  is (approximately) distributed as a uniform over  $\{0, \dots, m-1\}$

$\Rightarrow$  Define  $U_j := X_j/m$ ,  $U_j \approx \text{Uniform}[0, 1]$

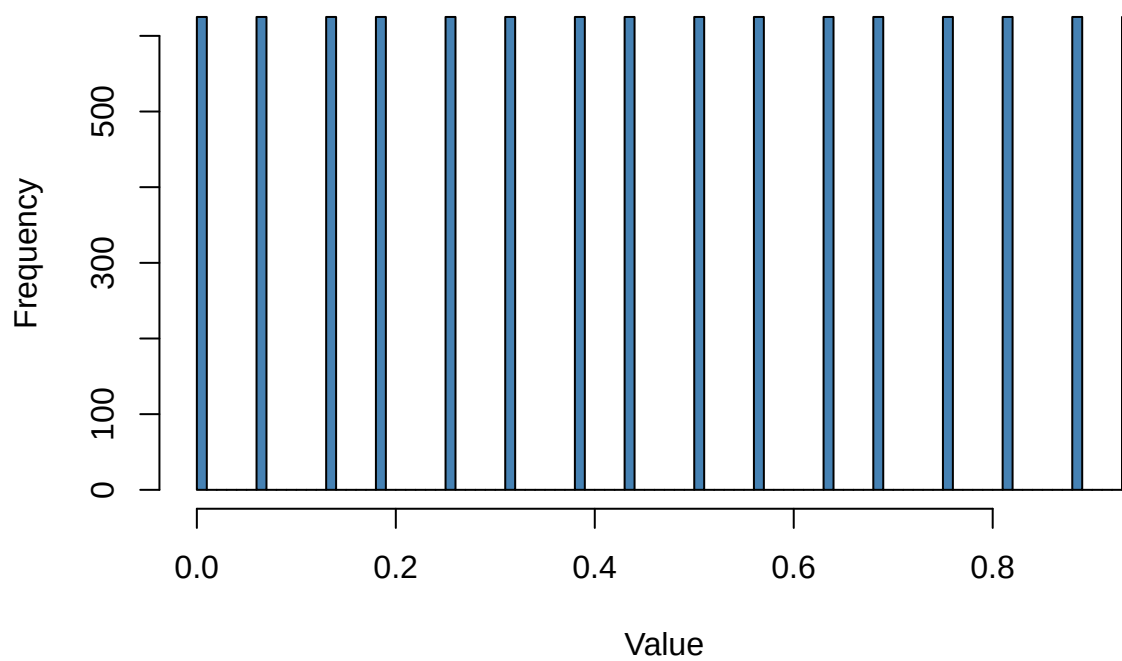
```
# 'yield' equivalence: a closure preserving the generator's internal state
lcg <- function(modulus, a, c, seed) {
  state <- seed
  function() {
    state <- (a * state + c) %% modulus
    return(state)
  }
}
```

```
mod <- 128
mult <- 15
increment <- 25
seed <- 1

samples1 <- numeric(10000)
gen <- function(x) {
  lcg(mod, mult, increment, x)
}

g <- gen(seed)
for (i in 1:10000) {
  temp <- g()
  samples1[i] <- temp / mod
}
```

```
# Plot histogram using R Base
hist(
  samples1,
  breaks = 100,
  col = "steelblue",
  main = "",
  xlab = "Value",
  ylab = "Frequency"
)
```



```
mod <- 2^31
mult <- 1103515245
increment <- 12345
seed <- 123

samples2 <- numeric(10000)

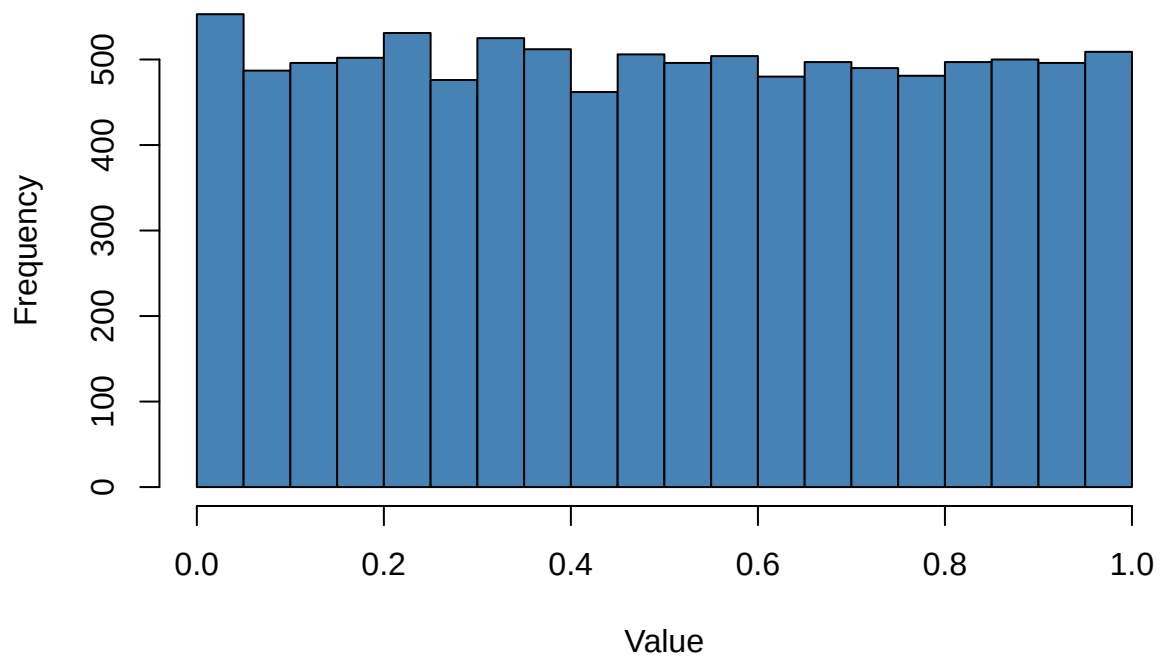
g <- gen(seed)
for (i in 1:10000) {
  temp <- g()
  samples2[i] <- temp / mod
}

# Plot histogram using R Base (Scott's rule for binning)
hist(
```

```

samples2,
breaks = "Scott",
col = "steelblue",
main = "",
xlab = "Value",
ylab = "Frequency"
)

```



## Compare two PRNGs using the Kolmogorov - Smirnov test

Given samples  $x_1, \dots, x_n$  let  $F_n(t) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[x_i < t]$  the empirical CDF.

The Kolmogorov-Smirnov test compares the empirical CDF with the expected one, i.e. tests

$$H_0 : \lim_{n \rightarrow \infty} F_n = F_0 \quad \text{vs.} \quad H_1 : \lim_{n \rightarrow \infty} F_n \neq F_0$$

Is based on the Kolmogorov-Smirnov distance

$$KS(F_n, F_0) = \sup_x |F_n(x) - F_0(x)|$$

```

# Kolmogorov-Smirnov test against uniform distribution U(0, 1)
ks.test(samples1, "punif")

```

```
## Warning in ks.test.default(samples1, "punif"): ties should not be present for
## the one-sample Kolmogorov-Smirnov test
```

```
##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: samples1
## D = 0.0625, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```
# Kolmogorov-Smirnov test against uniform distribution U(0, 1)
ks.test(samples2, "punif")
```

```
##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: samples2
## D = 0.0086363, p-value = 0.4449
## alternative hypothesis: two-sided
```

```
# Kolmogorov-Smirnov test against uniform distribution U(0, 1)
ks.test(runif(1000), "punif")
```

```
##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: runif(1000)
## D = 0.031241, p-value = 0.2832
## alternative hypothesis: two-sided
```

## The Inverse CDF method

To generate a random value from  $X$  we can:

1. Simulate  $U \sim \text{Uniform}[0, 1]$
2. Return  $F_X^{-1}(U)$

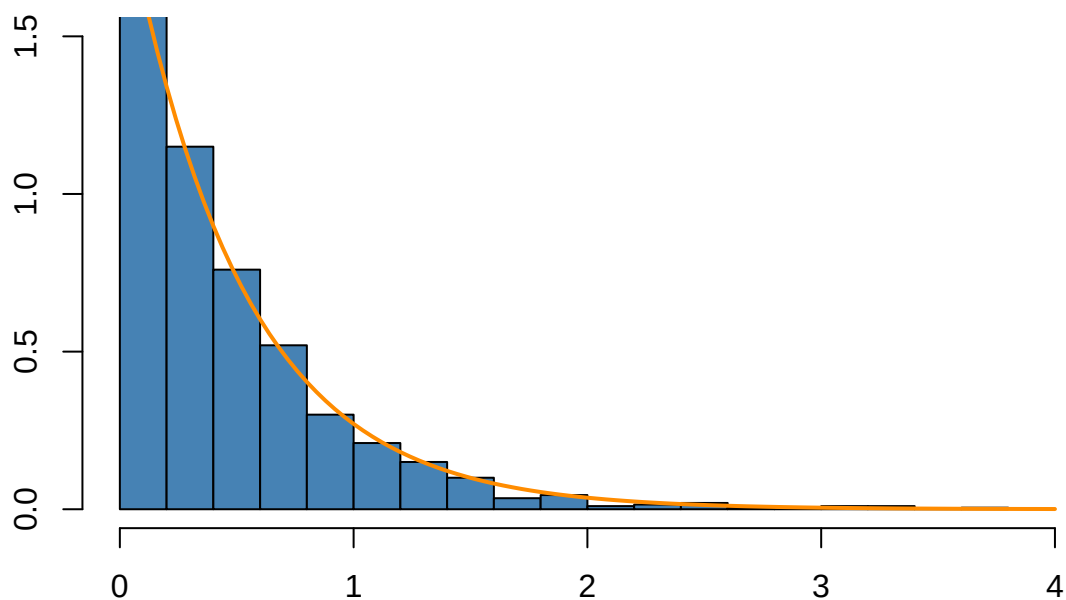
N.B. this works also for non continuous random variables!!

### Example 1 - Exponential distribution

$$X \sim \text{Exp}(\lambda) \quad \Rightarrow \quad F_X(x) = 1 - \exp(-\lambda x)$$

```
exponential_rng <- function(lambda, size = 1){
  u <- runif(size)
  -log(1 - u)/lambda
}

hist(exponential_rng(2,1000), breaks = "Scott",
  col = "steelblue", , probability=TRUE, xlim = c(0,4), ylim = c(0,1.5), main = '', xlab = '', ylab = ''
curve(dexp(x, 2), add = T, col = 'darkorange', lwd = 2)
```



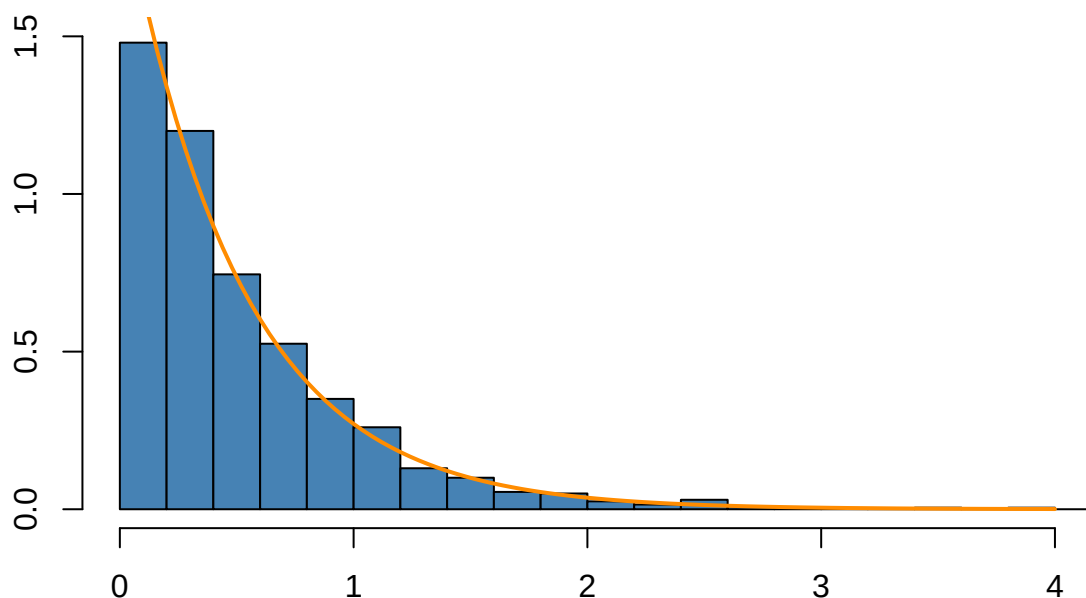
Observe that

$$1 - U \sim \text{Uniform}[0, 1]$$

so we can speed up the code writing

```
exponential_rng <- function(lambda, size = 1){
  u <- runif(size)
  -log(u)/lambda
}

hist(exponential_rng(2,1000), breaks = "Scott",
  col = "steelblue", probability=TRUE, xlim = c(0,4), ylim = c(0,1.5), main = '', xlab = '', ylab = '')
curve(dexp(x, 2), add = T, col = 'darkorange', lwd = 2)
```



## Example 2 - the truncated Normal distribution

We say that  $X$  follows a truncated normal distribution with parameters  $\mu, \sigma^2, a, b$  if its probability density function is

$$f_X(x) = \frac{1}{Z(\mu, \sigma^2, a, b)} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \mathbb{I}[x \in (a, b)] \propto \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \mathbb{I}[x \in (a, b)]$$

```
library(ggplot2)

# Truncated normal sampler via inverse CDF method
trunc_normal_rng <- function(mu, sigma, a, b, size) {
  u <- runif(size)
  z <- pnorm((b - mu) / sigma) - pnorm((a - mu) / sigma)
  qnorm(z * u + pnorm((a - mu) / sigma)) * sigma + mu
}

# Grid of points and theoretical PDF calculation
xx <- seq(-3, 10, length.out = 1000)
pdf_vals <- dnorm(xx, mean = 0, sd = 2) / (pnorm(10 / 2) - pnorm(-3 / 2))

# Generate sample data and create data frames for ggplot2
samples <- trunc_normal_rng(mu = 0, sigma = 2, a = -3, b = 10, size = 10000)
df_samples <- data.frame(x = samples)
```

```
df_pdf <- data.frame(x = xx, y = pdf_vals)

# Compute binwidth using Scott's rule
bw <- 3.5 * sd(samples) / (length(samples)^(1/3))

# Plot histogram with overlaid theoretical density
ggplot() +
  geom_histogram(
    data = df_samples,
    aes(x = x, y = after_stat(density)),
    binwidth = bw,
    fill = "steelblue",
    color = "black",
    alpha = 0.5
  ) +
  geom_line(data = df_pdf, aes(x = x, y = y), color = "darkorange", linewidth = 1.2) +
  theme_minimal() +
  labs(x = "x", y = "Density")
```

